

Assignment 2

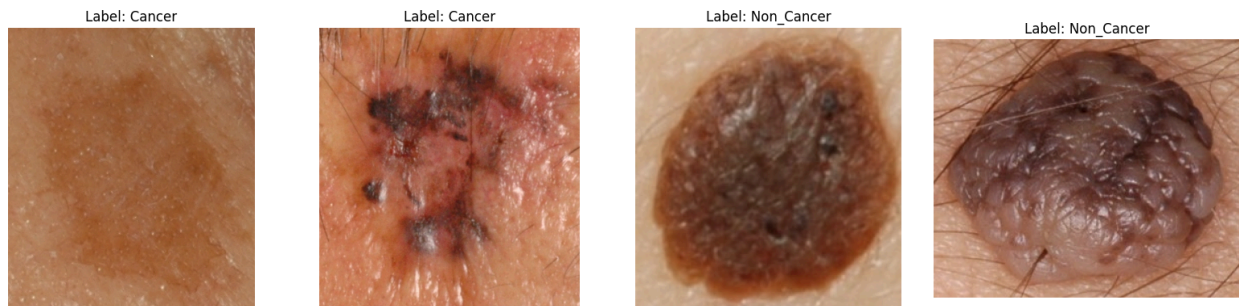
Kavin Ravi

Introduction

For this assignment, I will be attempting to classify a dataset of moles based on whether or not they are cancerous. This is a binary classification image dataset, for which I will use a standard Convolutional Neural Network, and later on use transfer learning to use a much larger pretrained model with a custom head.

Analysis

The first step was to get a feel for my dataset. The dataset itself was only around 240 images, which would look something like the following:



Something I noticed upon immediate cursory inspection was that the cancerous moles seemed to have a brown outline or ring around them, and didn't seem to have much dimension (the two lumpy/uneven ones were both non-cancerous).

Methodology

First off, I had to organize my data into training/testing directories. As it was, the dataset was split by class (cancer/non_cancer) and each subdirectory had a training/testing sub-subdirectory, which would be inconvenient for model training, so the first step was to organize the sub-subdirectories into the subdirectories "training" and "testing". The next step was to use data augmentation in order to both bolster the robustness of the model and to add more volume to the dataset. A deep convolutional neural network trying to fit on only 240 images is more or less a recipe for overfitting, so I planned to counter that possibility by augmenting my data to a total of approximately 800 images. Then, I rescaled (by dividing color values by 255) and split my data into the training and testing batches, and constructed my models.

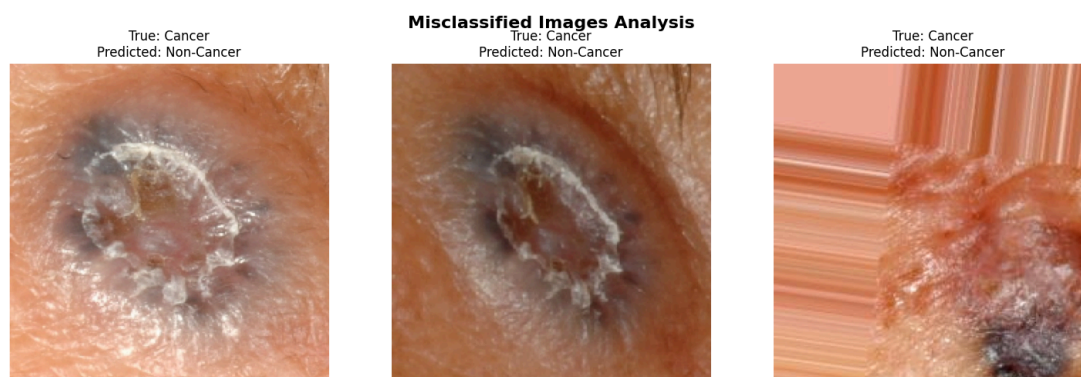
The first model was a basic deep convolutional neural network, with 5 convolutional layers, 3 max pooling layers, and around 2.5 million total parameters. It was trained using the Adam optimizer (learning rate $1e-3$) on 10 epochs and a batch size of 32. Dropout and L2 reg were implemented in order to combat overfitting.

The second model was the Xception model offered through tensorflow.keras.applications, a model with around 21.3 million total parameters after removing the existing head and adding an average pooling and dense layer. For this model, Early Stopping, Dropout, and L2 Reg were implemented in order to prevent overfitting, considering the size of the model.

Results

The results were certainly interesting. As cited in my README, I've been having some pretty annoying technical issues for the past couple weeks that have ruined my model training performance. These issues were resolved upon removing the tensorflow-metal package, as it apparently is notorious for broken builds and working incorrectly. In the case of my models, the first model performed poorly (as expected), settling at exactly 0.500 training and val accuracy. At first, I was terrified upon seeing this, as these were exactly the patterns displayed during the two week period during which tensorflow-metal was ruining my model training. But, upon closer inspection, the loss was still being iteratively reduced by each epoch, and considering this is a binary classification problem it's actually just very likely that the data augmentation was aggressive enough, and there were few enough samples, such that the network was unable to robustly predict classes during training and validation, but the regularization prevented it from overfitting. This is both a good and bad sign; on one hand, performance is lacking, on the other, overfitting is being avoided.

The transfer learning model was treated similarly, with no additional training code being implemented. The only additions to the existing ~20m param Xception model was some light L2 regularization and dropout, and early stopping as well. It achieved a final training accuracy of 66% and val accuracy of around 54%. At first glance, this may seem indicative of instant overfitting, but I notice the training accuracy decreasing to its final value of 66% and testing accuracy increasing to its final value of 54%, meaning that given some more training time it's very possible the two performances may have converged. I was able to get up to 93% training accuracy without any regularization, but the validation accuracy remained more or less the same, if not slightly worse. Nonetheless, it's important to examine when the model misclassified an image (which seems to be exactly half of the time in the validation set), and see if any patterns emerge:



Indeed, these misclassifications actually fall in line with what I predicted as cancer and non-cancer. Above, I observed that images showing cancerous moles usually weren't lumpy, and were flat. The model here classified lumpy moles as non-cancerous (as my initial observation would have indicated), when in truth they were cancerous. As for the last augmented image, this is what informs my hypothesis that the data augmentation was too strong. The image is nearly unrecognizable.

Conclusion

Overall, this was certainly an interesting experience. I was glad to have discovered the issues with tensorflow, though I'm still not sure how it didn't work properly on my PC, as that would have no reason to ever have tensorflow-metal installed on that device. The evidence that it's working properly even now is tenuous at best, because although perfect 50% accuracy makes sense in a vacuum, it doesn't make practical sense for every epoch to be getting that result. However, the transfer learning model performed perfectly fine, so it doesn't seem as though anything is inherently wrong with my system. I think the main problem came down to insufficient data. For such a deep neural network, with millions of parameters, convolutional layers, aggressive data augmentation, etc., 240 base images augmented into 600-800 (I tried both) was doomed to perform poorly from the start. There is also the possibility that the reason the transfer learning model didn't boost performance too much compared to other models is because skin cancer classification isn't something any transfer learning model like this would've been trained on, compared to something like food, animals, objects, or anything more common to classify on.